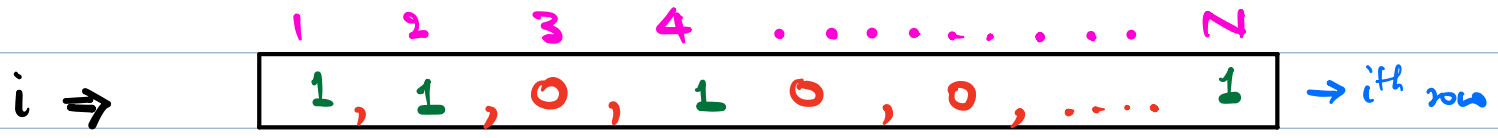
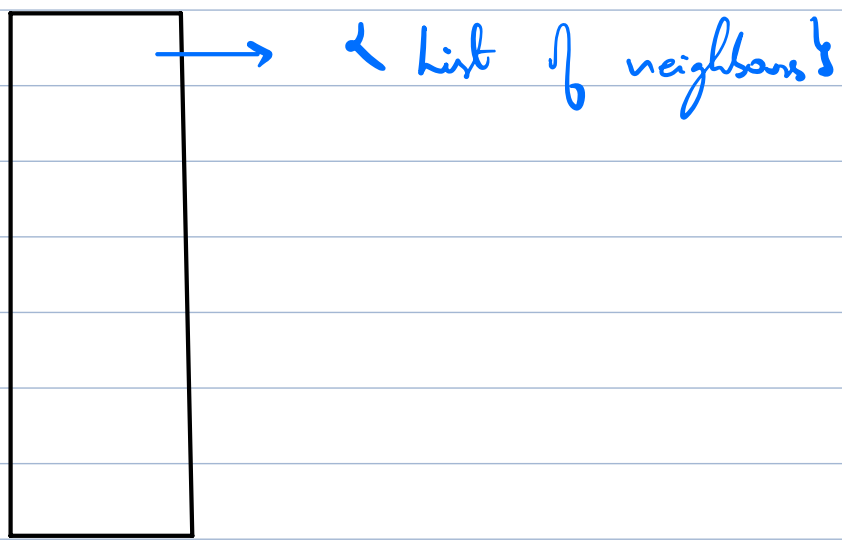
M[N][N]

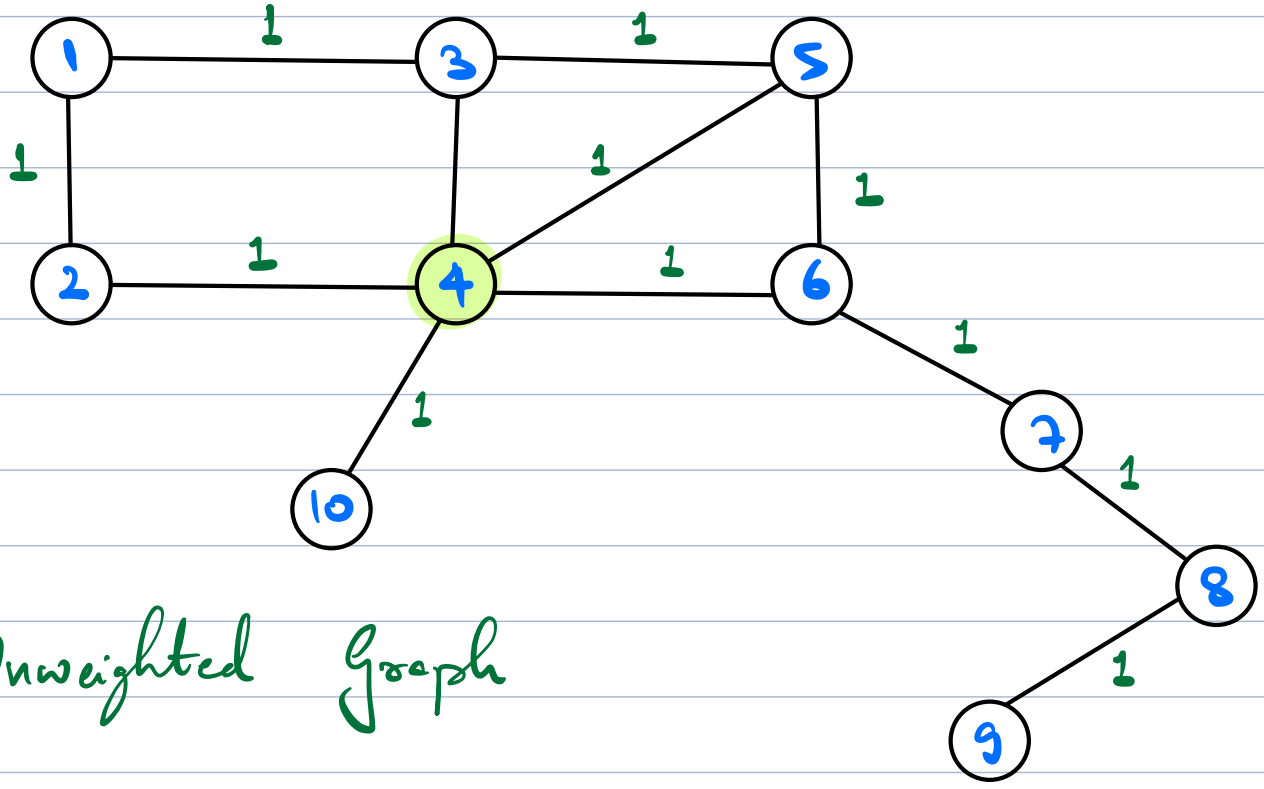
M[i][j] ⇒ Whether there exists an edge b/w the node i & node j





Length of a path from i to j = Sum of weights of Edges b/w i to j

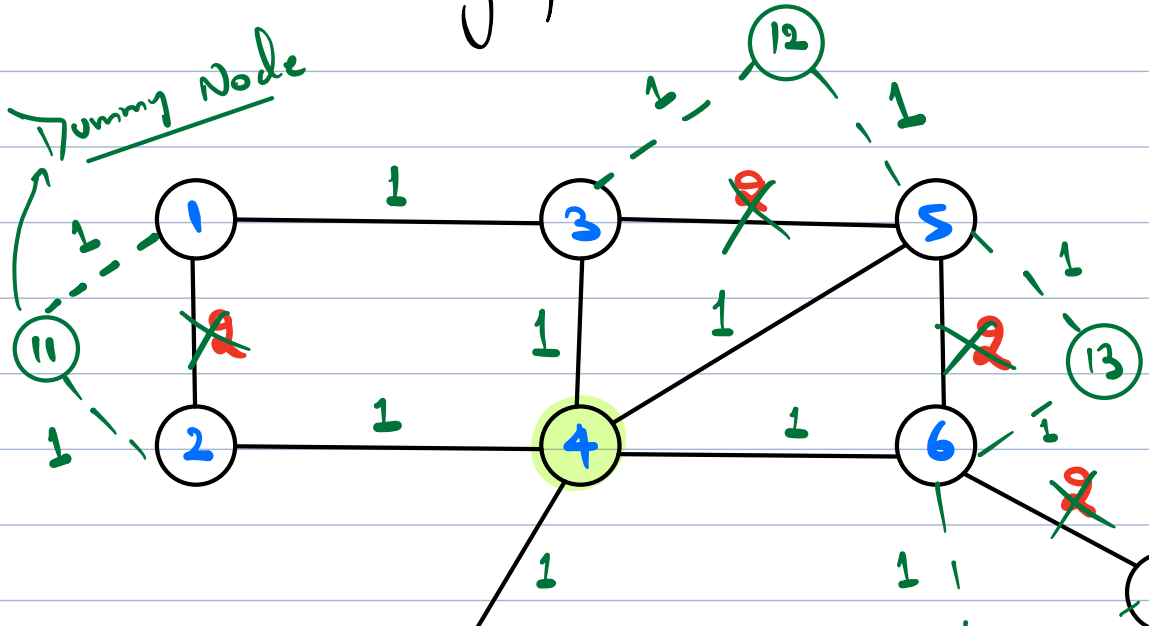
If weights of all Edges = 1 (Unweighted)

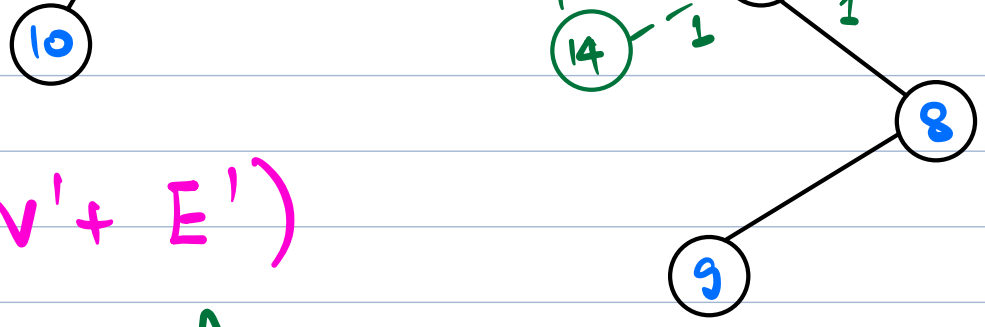


Unweighted Graph

Shortest distance from a given source $\Rightarrow$ BFS

Given a graph with N nodes.
Weights of Edges = {1, 2}. find shortest distance from a given source to every other node in graph





$T.C. = O(V' + E')$

dist: [(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10) 11, 12, 13, 14]

$V' = V + E$

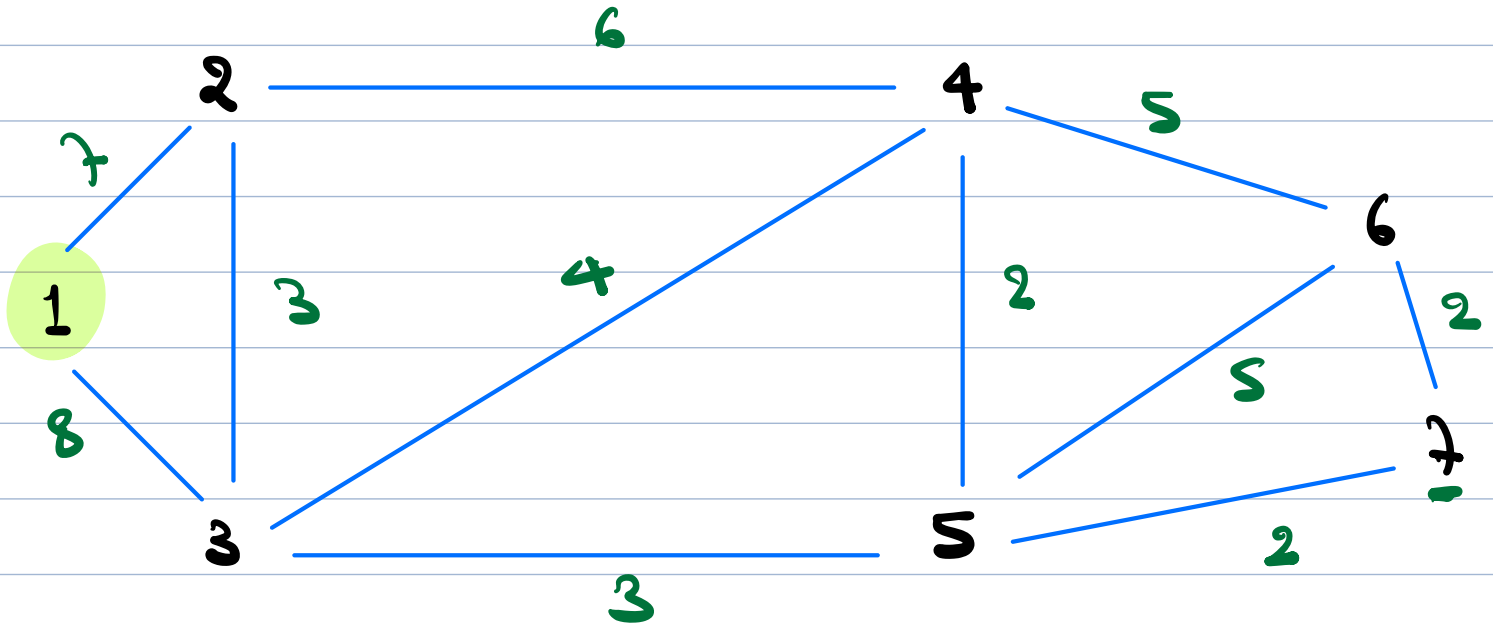
$E' = 2E$

$T.C. = O(V + E + 2E)$

$= O(V + 3E)$

$= O(V + E)$

Dijikstra's Algorithm



1	2	3	4	5	6	7
0	7	8	12	11	15	13

⟨ dist, Node ⟩

~~7, 2~~
~~8, 3~~
~~13, 4~~
~~12, 4~~
~~11, 5~~
~~16, 6~~
~~13, 7~~
~~15, 6~~

1) 7, 2 → ~~1~~, ~~3~~, 4
~~4~~ ~~13~~ 13

2) 8, 3 → ~~1~~, ~~2~~, 4, 5
~~16~~ ~~11~~ 12 11

3) 11, 5 → ~~3~~, ~~4~~, 6, 7
~~13~~ 16 13

4) 12, 4 → ~~2~~, ~~6~~, ~~3~~, ~~5~~

~~5) 13, 4~~

6) 13, 7 → 6, ~~5~~

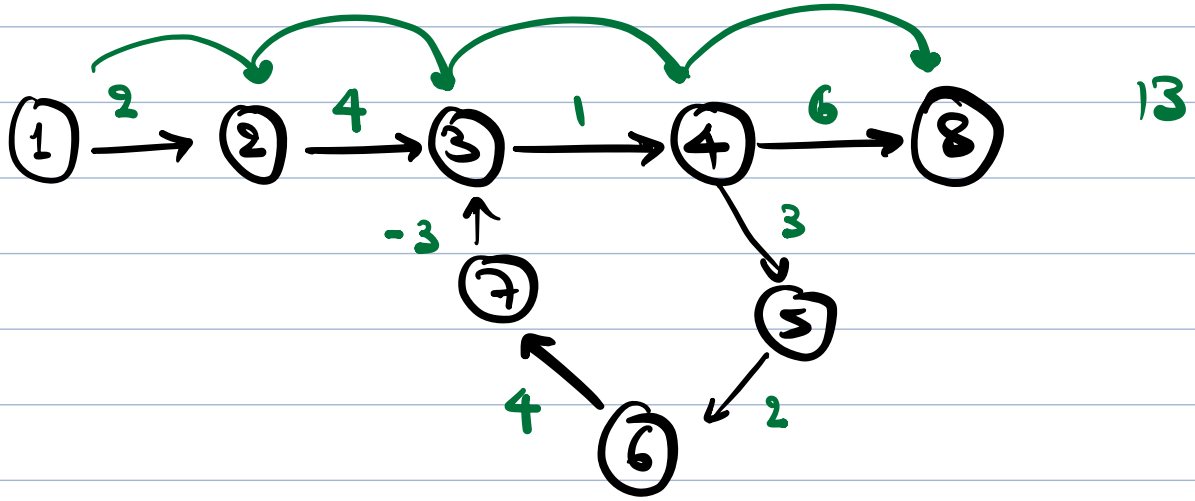
7) 15, 6 → ~~4~~, ~~2~~, ~~3~~

~~8) 16, 6~~

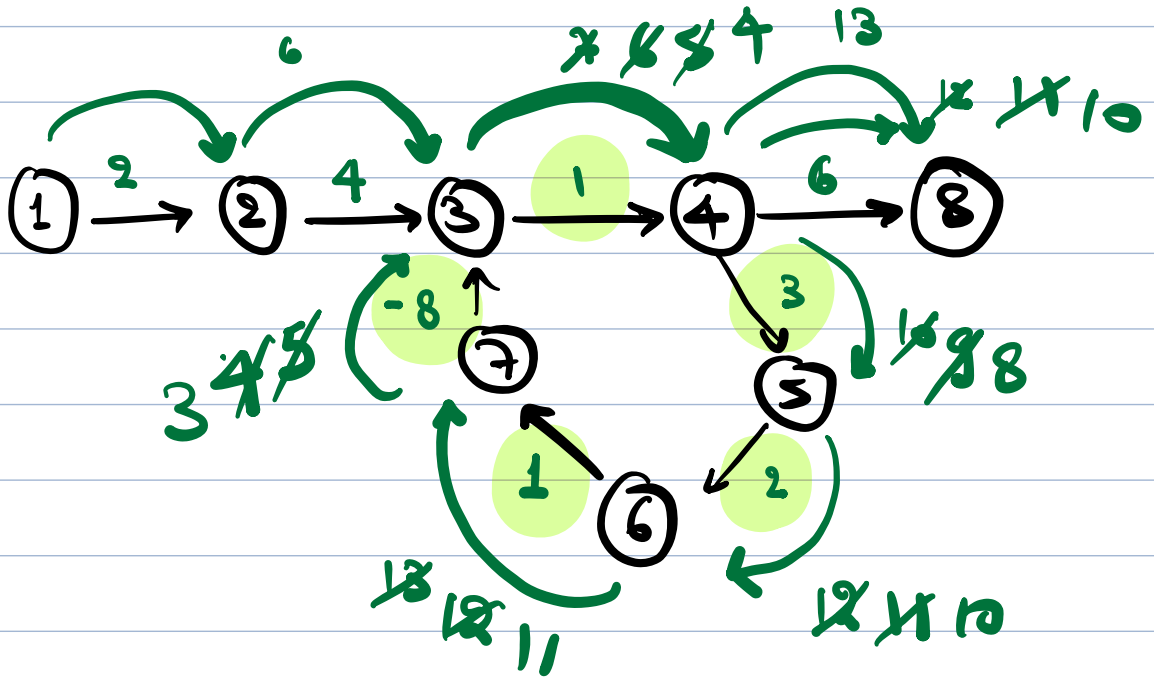
Code

```
                    dist    node
MinHeap<Pair<int, int>>mh;

dist[N+1] = {∞};

dist[source] = 0;

for (all nodes u connected to source) {
        mh.insert(W_source-u, u);
        dist[u] = W_s-u;
```

```
}

while ( ! mh.isEmpty() ) {

    Pair<int, int> p = mh.getMin();

    int u = p.second();
    int d = p.first();

    if ( d == dist[u] ) {
        for (all nodes v connected to u) {
            distance = d + W_{u-v};
            if ( distance < dist[v] ) {
                dist[v] = distance;
                mh.insert( new Pair ( distance, v ) );
            }
        }
    }
}
```



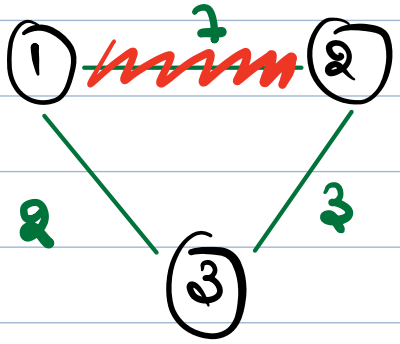
13



Negative Weight Cycle ⇒ No shortest distance

Bellman Ford Algorithm

⇒ Update / Relax an Edge (Path)



1 - 2	~~7~~ 5
1 - 3	2
3 - 2	3

Compare D_{1-2} with D from 1 to 2 via 3

$= D_{1-3} + W_{3-2}$

$= 2 + 3 = 5$

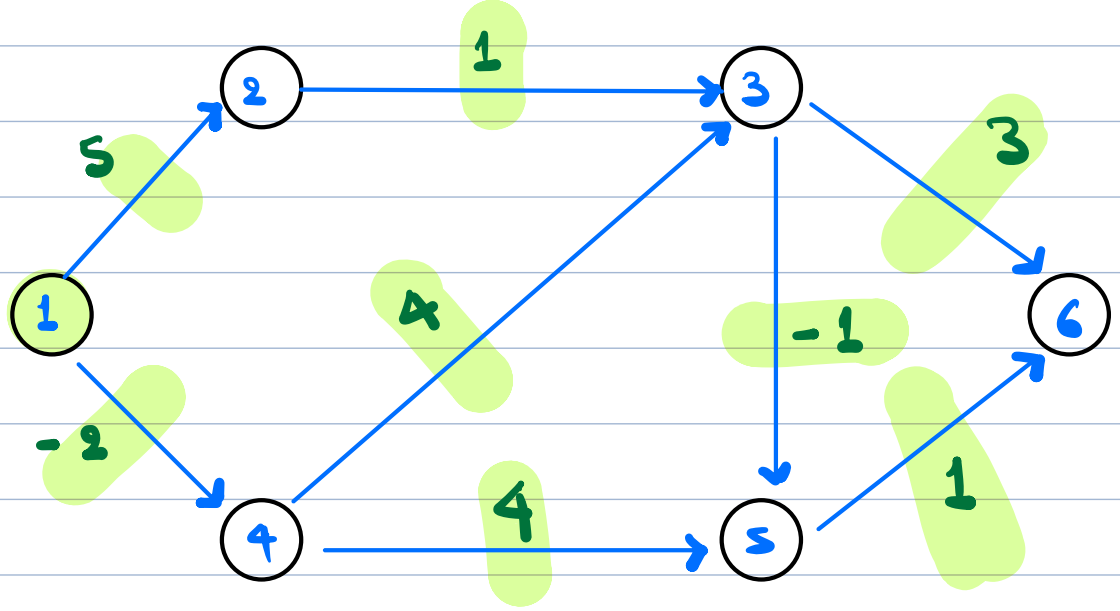
Idea

⇒ Relax every edge of the graph (N-1) times.

⇒ To relax an edge from node u to v.

Calculate distance from u to v via every other node & check if the distance is less than current distance.

Eg



1	2	3	4	5	6
0	5	2	-2	2	3

1 → 2 source — 2 via 1

X

$d[1] + W_{1\to 2} = \not{0} + 5 = 5$
$6 + 5 = 5$

2 → 3 $1 \longrightarrow 3$ via 2

X

$d[2] + W_{2\to 3}$
$= \not{8} + 1 = 6$
$5 + 1 = 6$

4 → 5 $1 \longrightarrow 5$ via 4

✓

$d[4] + W_{1\to 5}$

$\infty + 4 = \infty$
$-2 + 4 = 2$

3 → 6 $1 \longrightarrow 6$ via 3

X

$d[3] + W_{3\to 6}$
$6 + 3 = 9$
$6 + 3 = 9$

5 → 6 $1 \longrightarrow 6$ via 5

✓

$d[5] + W_{5\to 6}$
$\infty + 1 = \infty$
$2 + 1 = 3$

3 → 5 ✗

1 → 5 via 3

d[3] + $w_{3\to5}$

6 + (-1) = 5

6 + (-1) = 5

4 → 3 ✓

1 → 3 via 4

d[4] + $w_{4\to3}$

∞ + 4 = ∞

-2 + 4 = 2

1 → 4 ✗

1 → 4 via 1

d[1] + $w_{1\to4}$

0 + (-2) = -2

E[i][0] = s E[i][1] = d E[i][2] = w

```
int[] minDistance (N, E, Edges[E][3], source) {

    dist[N+1] = ∞;
    dist[source] = 0;

    for (i = 0; i < (N-1); i++) {                    O(N)
        bool stop = True;
        for (j = 0; j < Edges.size(); j++) {         O(E)
            s = Edges[j][0];
            d = Edges[j][1];
```

```
w = edges[j][2];

// source → d via s
if (dist[s] + w < dist[d]) {
    stop = false;
    dist[d] = dist[s] + w;
}
}
if (stop == True) {
    break;
}
}
return dist;
}
```

Check for ∞

T.C. = $O(N \times E)$

Floyd Warshall Algorithm.

All pair shortest distance algo.

M[v][v],

M[i][j] ⇒ denote shortest dist from i to j

ayushsharma@scebe.in